

ACSA — Fitxers històrics

Documentació tècnica del frontend

Versió 0.1.0 · Generada automàticament

Aquest document és la font única de veritat per a desenvolupadors, revisors, DevOps i futurs mantenidors del projecte acsa-frontend. El backend API resideix en un repositori separat (Automatització/backend); aquesta documentació cobreix exclusivament la capa d'interfície React.

(La taula de continguts es genera al final del document.)

Propòsit

ACSA Frontend és una aplicació web d'una sola pàgina (SPA) per digitalitzar, classificar amb IA, revisar humànement i arxivar documents PDF històrics de l'organització ACSA.

Domini de negoci

- Gestió del cicle de vida de documents històrics en PDF
- Classificació automàtica mitjançant OCR i Google Gemini
- Revisió humana abans de l'arxivament definitiu
- Detecció de duplicats i comparació de similitud
- Processament de correus electrònics com a font documental

Usuaris previstos

- Operadors d'escaneig i càrrega de documents
- Revisors documentals que validen noms i resums proposats
- Administradors que configuren carpetes, claus API i processos batch

Capacitats principals

- Flux principal en 4 passos: Escàner → Classificador → Revisió → Documents
- Eines: Comparador de duplicats, Configuració
- Administració: Anàlisi batch, Edicions al sistema de fitxers, Correus

Stack tecnològic

Capa	Tecnologia
UI	React 18.3 + TypeScript 5.6
Build	Vite 5.4
Routing	react-router-dom 6.28
Estat servidor	@tanstack/react-query 5.60
Notificacions	react-hot-toast
PDF	pdfjs-dist (actius a public/pdfjs/)
Producció	nginx Alpine (Docker multi-stage)

Arquitectura d'alt nivell

El navegador executa la SPA compilada. Les peticions API passen per /api (proxy Vite en dev, nginx en producció) cap al backend Python/FastAPI (inferit) al port 8000. L'autenticació es fa amb capçalera X-API-Key, no amb sessions d'usuari.

Integracions externes

- Backend ACSA API (REST JSON)
- Google Gemini (configurat al backend via pantalla Configuració)
- Sistema de fitxers local/xarxa (carpetes d'entrada i sortida)

Model de desplegament

- Desenvolupament: npm run dev (port 5173) amb proxy a localhost:8000
- Docker: imatge nginx servint dist/ + entrypoint que injecta config.js
- docker-compose: servei web a port 8080, xarxa externa acsa

2. Visió general de l'arquitectura

Arquitectura per capes

- Presentació: `src/pages/` + `src/components/`
- Estat: React Query (cache servidor) + Context (treball classificador)
- Accés a dades: `src/api/client.ts`
- Configuració: `src/config.ts` + `public/config.js` (runtime)
- Utilitats: `src/utils/`, `src/hooks/`, `src/constants/`

Límits de domini

Cada pàgina correspon a una capacitat de negoci. El client API centralitza totes les interaccions HTTP. Els tipus a `src/api/types.ts` reflecteixen l'esquema del backend i s'han de mantenir sincronitzats manualment.

Cicle de vida d'una petició

```

Usuari → Component React → React Query / Mutation
      → client.ts (buildHeaders + fetch)
      → /api (proxy nginx/Vite)
      → Backend FastAPI
      → JSON { status, data } o cos directe
      → unwrapEnvelope → actualització UI + toast
  
```

Processament en segon pla

Els treballs d'anàlisi (jobs) s'executen al backend. El frontend fa polling cada 2s via `useJobPolling` fins a estat terminal (completed/failed/cancelled). Després de completar l'anàlisi, `ClassificadorJobContext` dispara automàticament `assignDocuments()`.

Diagrama d'arquitectura del sistema

```

[Navegador SPA]
| HTTPS/HTTP
v
[nginx :80] -- /api --> [Backend :8000]
|               |
|-- config.js    |-- OCR + Gemini
|-- assets estàtics  |-- SQLite/FS (inferit)
v
[React + React Query]
  
```

Flux del classificador

```

ClassificadorPage.startAnalyze()
  → POST /jobs/analyze (source=inbox)
  → polling GET /jobs/{id}
  → status=completed
  → POST /assign (inbox → archive, require_review=true)
  → documents amb status=revisio
  
```

Graf de dependències de mòduls

```

main.tsx → App.tsx → Layout + Pages
App.tsx → ClassificadorJobContext → useJobPolling → api/client
Pages → api/client → config.ts
PdfPreview → api/client (fetch blob PDF)
  
```


3. Documentació funcional

3.1 Escaneig i càrrega (UploadPage)

Propòsit: Ingressar PDF nous a la safata d'entrada (_PENDENTS).

Flux: Drag-and-drop → validació PDF → POST /files/upload/batch → llista local d'història.

Components: UploadPage, uploadBatch(), getSettings()

Errors/edge cases: Rebutja fitxers no-PDF; mostra errors ApiError.

3.2 Classificació IA (ClassificadorPage)

Propòsit: Analitzar PDF de la safata amb OCR+Gemini i assignar-los a revisió.

Flux: startAnalyzeJob → polling → assignDocuments automàtic.

Components: ClassificadorJobContext, JobProgressPanel, useJobPolling

Errors/edge cases: 401 → missatge 'Accés no autoritzat'; cancel·lació via DELETE /jobs/{id}.

3.3 Revisió humana (RevisioPage)

Propòsit: Validar documents amb status=revisio abans de l'arxivament.

Flux: listDocuments → selecció → PdfPreview → edició nom/resum → approve o delete.

Components: PdfPreview, TablePagination, DeleteDocumentButton

Errors/edge cases: Concurrencia: dues pestanyes editant el mateix document.

3.4 Arxiu de documents (DocumentsPage)

Propòsit: Consultar, filtrar, ordenar i moure documents aprovats (status=ok).

Flux: Filtres servidor/client híbrids → paginació → moveDocument.

Components: useDocumentFilterOptions, matchDocumentFilters, folderSuggestions

Errors/edge cases: Filtres complexos carreguen tots els documents (rendiment).

3.5 Configuració (SettingsPage)

Propòsit: Gestionar carpetes i claus Gemini al backend.

Flux: GET /settings → formulari → PUT /settings.

Components: SettingsPage, updateSettings()

Errors/edge cases: Claus buides no s'envien (undefined) per no esborrar-les.

3.6 Comparador (ComparadorPage)

Propòsit: Detectar duplicats o similituds contra l'arxiu.

Flux: POST /compare/scan amb FormData.

Components: FilePdfPreview, compareFile()

Errors/edge cases: Veredictes: duplicate | similar | none.

3.7 Anàlisi administrativa (AnalisiPage)

Propòsit: Control granular de treballs batch (carpetes, límits, dry-run).

Flux: Formulari avançat → startAnalyzeJob amb paràmetres complets.

Components: AnalisiPage, listFolders()

Errors/edge cases: Jobs llargs sense timeout al frontend.

3.8 Edicions al filesystem (EdicionsPage)

Propòsit: Aplicar o revertir canvis de nom al disc.

Flux: POST /apply o POST /revert.

Components: applyRenames(), revertRenames()

3.9 Correus (CorreusPage)

Propòsit: Pipeline paral·lel per documents provinents de correu.

Flux: listEmails → anàlisi → assign → revisió/aprovació.

Components: startEmailAnalyzeJob(), assignEmails()

Errors/edge cases: Estat independent dels documents PDF.

4. Referència per a desenvolupadors

src/api/client.ts

Client HTTP central. Exporta `ApiError`, `buildHeaders()`, `isUnauthorizedError()` i funcions per cada endpoint. La funció `request()` desenvolupa `{status,data}`, gestiona toasts i errors HTTP.

src/api/types.ts

Interfícies TypeScript que reflecteixen l'esquema del backend. Mantenir sincronitzat amb Automatització/backend.

src/config.ts

`getApiKey(): runtime config` → `VITE_API_KEY`. `getApiBaseUrl(): runtime` → `VITE_API_URL` → `/api`.

src/contexts/ClassificadorJobContext.tsx

Provider global per al treball del classificador. Gestiona `jobId`, polling, auto-assignació post-completed i errors 401.

Hooks

Hook

`useJobPolling`
`useDebounceValue`
`useDocumentFilterOptions`
`useDocumentListTotal`

Responsabilitat

Polling recursiu de jobs amb backoff en error
 Debounce per camps de cerca
 Opcions de filtre des de documents
 Comptador total amb filtres

Components clau

Component

`Layout`
`PdfPreview`
`JobProgressPanel`
`FilterAutocompleteInput`
`TablePagination`

Responsabilitat

Shell amb sidebar i banner de job global
 Fetch blob PDF amb `AbortController` i revocació URL
 Progrés i cancel·lació de jobs
 Autocomplete accessible (ARIA)
 Paginació reutilitzable

Especificació OpenAPI 3.1 disponible a [docs/openapi.yaml](#). Totes les rutes requereixen X-API-Key excepte /health.

Endpoints documentats

Mètode	Ruta	Funció client
GET	/health	getHealth()
POST	/files/upload/batch	uploadBatch()
GET	/documents	listDocuments()
PATCH	/documents/{id}	updateDocument()
DELETE	/documents/{id}	deleteDocument()
POST	/documents/{id}/move	moveDocument()
GET	/documents/{id}/file	documentFileUrl()
POST	/jobs/analyze	startAnalyzeJob()
GET	/jobs/{id}	getJob()
DELETE	/jobs/{id}	cancelJob()
POST	/assign	assignDocuments()
GET/PUT	/settings	getSettings/updateSettings
POST	/compare/scan	compareFile()
POST	/apply	applyRenames()
POST	/revert	revertRenames()
GET	/folders	listFolders()
GET	/emails	listEmails()
POST	/emails/assign	assignEmails()
POST	/jobs/email-analyze	startEmailAnalyzeJob()

Exemple cURL

```
curl -H "X-API-Key: LA_CLAU" \  
"http://localhost:8000/documents?status=revisio&limit=10"
```

Errors HTTP

- 401: Clau API invàlida → `isUnauthorizedError()`
- 4xx/5xx: `parseError()` extreu detail de FastAPI
- 204: sense cos (delete)

DocumentOut (principal)

id, status (ok|revisio), proposed_name, original_name, company, company_folder, doc_type_ca, final_date, summary, target_folder, language, processed_at, duplicate, compare.

JobOut

id, type, status (pending|running|completed|failed|cancelled), progress {processed,total,current_file,status_counts}, error, result.

SettingsOut

input_folder, output_folder, gemini_api_key (enmascarat), gemini_model, gemini_configured.

EmailOut

Paral·lel a DocumentOut amb camps de correu: subject, sender_email, sent_at.

Diagrama ER (conceptual)

Document (1) --- status: ok revisio
Job (N) --- processa --- Document
Email (N) --- pot generar --- Document
Settings (1) --- configura --- Backend

7. Configuració

Variable	On	Per defecte	Obligatori
VITE_API_URL	.env / build Docker	/api	No
VITE_API_KEY	.env / build Docker	(buit)	Recomanat
API_KEY	Docker runtime	VITE_API_KEY	Prod: Sí
API_UPSTREAM	Docker runtime	http://api:8000	Sí en compose
WEB_PORT	docker-compose	8080	No

config.js (runtime)

```
window.__ACSA_CONFIG__ = { apiKey: "...", apiUrl?: "..." };
```

En Docker, entryptpoint.sh escriu apiKey a l'arrencada. Les variables Vite queden fixades al build; la clau API es pot injectar sense rebuild.

Comportament en fallada

- Sense API_KEY: peticions sense capçalera → probable 401
- Backend no disponible: nginx arrenca igual (resolver dinàmic)
- Proxy dev sense VITE_API_KEY: cal configurar manualment

8. Guia de desplegament

Desenvolupament

```
npm ci
npm run dev
# Backend a localhost:8000
# .env: VITE_API_KEY=...
```

Docker

```
docker compose build
docker compose up -d
# http://localhost:8080
```

Ordre d'inicialització

- 1. Xarxa Docker externa 'acsa' ha d'existir
- 2. Backend API en la mateixa xarxa
- 3. entrypoint.sh: config.js + nginx upstream
- 4. nginx serveix SPA + proxy /api

CI/CD

No hi ha pipeline CI configurat en aquest repositori (assumpció verificada: no `.github/workflows`).

Health checks

`getHealth()` existeix al client però no s'usa a la UI. Recomanable afegir endpoint de readiness al contenidor.

Rollback

Desplegar imatge anterior de Docker. Els assets són immutables dins la imatge.

9. Errors comuns i trampes

Problema	Causa	Síntoma	Solució
Rutes admin sense protecció	Només ocultes al menú	Qualsevol usuari amb accés a la URL	Afegir guards o auth al backend
Clau API al client	config.js visible al navegador	Inspeccionar xarxa/font	Rotació de claus; claus restringides
DocumentsPage fetch total	Filtres clients actius	Lentitud amb milers de docs	Filtratge servidor complet
Polling sense timeout	Jobs penjats	UI bloquejada indefinidament	Timeout màxim al hook
Tipus desincronitzats	Canvis al backend	Errors runtime TS/API	Contract testing
Playwright no a package.json	Scripts de layout	npm install manual	Afegir com a devDependency
BASE URL al import	getApiBaseUrl() a mòdul load	Canvis runtime ignorats	Lazy resolution (baixa prioritat)

10. Revisió de seguretat

- Autenticació: només API key compartida, sense usuaris ni RBAC al frontend
- Autorització: inexistent a nivell UI; rutes /admin/* accessibles directament
- Secrets: VITE_API_KEY es pot incloure al bundle de build; preferir runtime config.js
- XSS: React escapa per defecte; PdfPreview usa blob URLs (baix risc)
- CSRF: no aplicable (API key, no cookies de sessió)
- Càrrega de fitxers: validació MIME/nom .pdf al client; validació servidor necessària
- Rate limiting: no implementat al frontend
- Logging: no es registren claus; toasts mostren missatges d'error genèrics

Recomanacions: autenticació per usuari, protecció de rutes admin, CSP headers a nginx, no exposar claus Gemini al client (ja gestionades al backend).

11. Revisió de rendiment

Àrea	Impacte	Recomanació
fetchAllDocuments	Alt amb arxius grans	Paginació servidor per tots els filtres
PdfPreview blob	Mitjà (memòria)	Revocació URL ja implementada correctament
React Query staleTime 30s	Baix	Adequat per llistes
Polling 2s	Baix-Mitjà	WebSocket o SSE si escala
public/pdfjs assets	Alt (mida imatge)	CDN o lazy load si cal
DocumentsPage re-renders	Mitjà	Memoització de files de taula

12. Estratègia de proves

- No hi ha framework de tests unitaris (Vitest/Jest) configurat
- Scripts Playwright ad hoc: test-revisio-table-height.mjs, test-documents-table-height.mjs
- Scripts de verificació API: verify-document-list-total.mjs, verify-folder-suggestions.mjs
- lint: només tsc --noEmit

Recomanacions: Vitest per utils/hooks, Testing Library per components, Playwright E2E al CI, contract tests amb openapi.yaml.

13. Deute tècnic

Element	Severitat	Esforç
0. Taula de continguts		
0. Taula de continguts	Alta	Mitjà
1. Resum executiu	Alta	Baix-Mitjà
2. Visió general de l'arquitectura	Mitjana	Mitjà
3. Documentació funcional	Baixa	Baix
4. Referència per a desenvolupadors	Mitjana	Baix
5. Referència API	Baixa	Alt
14. Full de ruta de millores		
6. Models de dades	Mitjana	Baix
7. Referència per a desenvolupadors	Baixa	Baix
8. Afegir README.md amb enllaç a aquest PDF		
9. Script nom docs: codebase.ca al package.json		
10. Health check al Layout		
11. Revisió de seguretat		
12. Eliminar PlaceholderPage si no cal		
Millores mitjanes		
13. Veste i tests de useJobPolling i matchDocumentFilters		
14. Full de ruta de millores		
• Playwright com a devDependency amb CI		
• Refactor de DocumentsPage en subcomponents		

Refactorització a llarg termini

- Autenticació per usuari amb JWT i rols
- Generació automàtica de tipus des d'OpenAPI del backend
- SSE/WebSockets per progrés de jobs
- Internacionalització (i18n) si cal castellà/anglès